`${parameter:?word}`

If the parameter is empty or unset, the expansion makes the script erroneous while it takes the value of the parameter if there are values assigned to it.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo:?"parameter is empty"}
bash: foo: parameter is empty
[me@linuxbox ~]$ echo $?
1
[me@linuxbox ~]$ foo=bar
[me@linuxbox ~]$ echo ${foo:?"parameter is empty"}
bar
[me@linuxbox ~]$ echo $?
0
```

`${parameter:+word}`

Here, if the parameter is unset or empty, the result from the expansion is nothing. Though if there are values, the word is then substituted for the parameter; though the value of parameter remains unchanged.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo:+"substitute value if set"}
[me@linuxbox ~]$ foo=bar
[me@linuxbox ~]$ echo ${foo:+"substitute value if set"}
substitute value if set
```

Expansions That Return Variable Names

Shell has the ability to give desired results from the name of the variables using:

`${!prefix*}`

`${!prefix@}`

This expansion returns the name of all existing variables that begins with a prefix. Here is an example of all variables that begin with the word BASH.

```
[me@linuxbox ~]$ echo ${!BASH*}
BASH BASH_ARGC BASH_ARGV BASH_COMMAND BASH_COMPLETION BASH_COMPLETION_DIR BASH_LINENO BASH_SOURCE BASH_SUBSHELL BASH_VERSINFO BASH_VERSION
```

String Operations

For strings, there is quite a large set to operate them in a shell and ideally suited for pathnames operations. `${#parameter}` expands with string length by parameter. Though it is a string, if used in combination with * or @; it can result